

Negotiated Local Plan Repair for Multi-Robot Warehouse Pathfinding

Sreenitya Thatikunta

October 2026

Abstract

Robots in a grid warehouse carry items from pickup cells to delivery stations along collision-free paths from prioritized Space-Time A* (ST-A*). While they execute these paths, robots break down, cells become blocked and robots receive urgent tasks. We repair the joint plan *locally*. Affected robots negotiate over a range-limited message bus with the smallest useful set of neighbours, and the global planner is never re-run. Over 1,700 simulated runs (5–50 robots, 0–15% dynamic obstacle density, 10 seeds each) there were no collisions, and every task was completed by every repair strategy except one gridlocked run of the no-negotiation ablation. Local repair changed $5\text{--}7\times$ fewer plans per disruption than replanning from scratch, at equal or lower total time. A stress study then identifies the settings in which robots *fail* to finish their tasks: disruptions that cut goals off, gridlock without negotiation, and emergency deadlines that become infeasible in dense fleets.

1 Problem

A team of n robots moves on a 4-connected grid. In each time step a robot moves to a neighbouring free cell or waits. Two robots may not occupy the same cell at the same time (vertex conflict) or swap cells (edge conflict). Each robot has a list of tasks (go to a pickup cell, then to a delivery station) and returns to its own dock after the last one. During execution three kinds of disruption occur:

1. **breakdown**: a robot stops, for 5–20 steps or permanently;
2. **blockage**: a free cell becomes an obstacle, for 10–40 steps or permanently;
3. **emergency task**: a robot receives a pickup–delivery task that goes before its others.

The assignment requires that disrupted robots repair their plans by communicating with nearby robots, that global planning is never re-run from scratch, and that as few robots as possible change their plans. We measure the total time to complete all tasks (sum of costs, SoC), the number of robots whose plans change per disruption, and how both scale with the number of robots and the density of dynamic obstacles.

2 Environment

Map. A Kiva-style warehouse of 18×33 cells (Figure 1): 5×5 shelf blocks of 2×5 cells separated by one-cell aisles, with a free perimeter corridor. It has 344 free cells, 150 pickup cells (next to a shelf), 14 delivery stations on the side walls, and 58 private docks on the top and bottom rows. Docks are dead-end bays, so robots never drive through each other’s docks.

Tasks. Each robot gets 3 random tasks, ordered greedily by nearest pickup. Picking up and dropping off take no extra time.

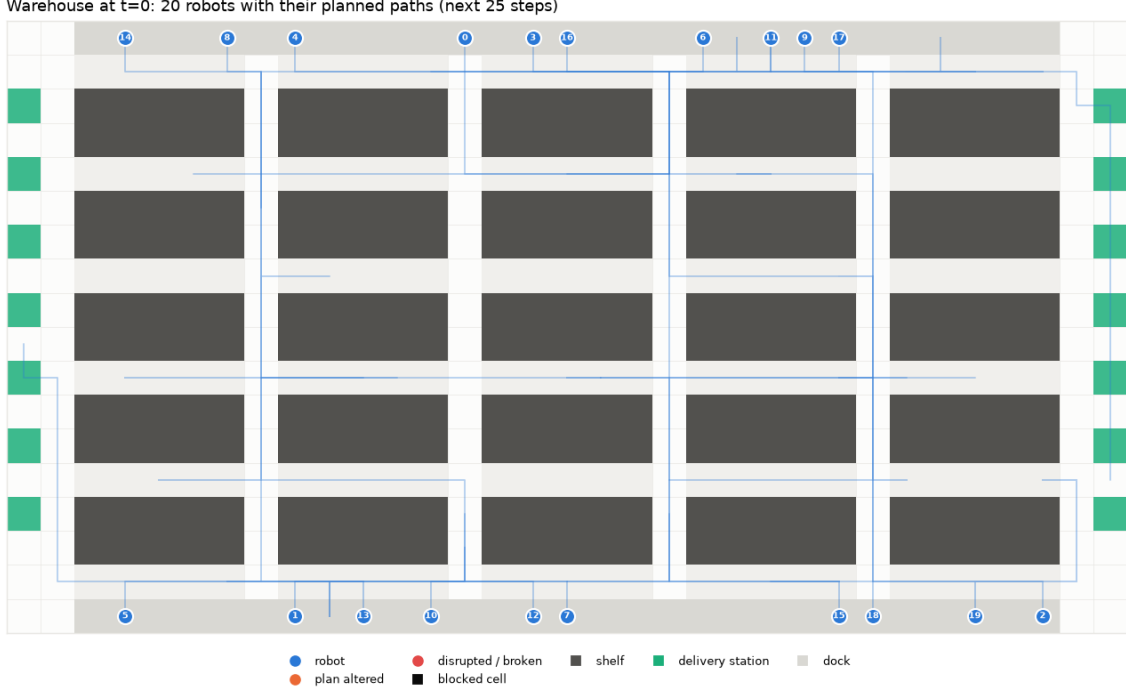


Figure 1: The warehouse at $t = 0$ with 20 robots and the next 25 steps of their planned paths.

Disruptions. Events are generated up front from the seed, so every strategy faces the same schedule. They happen in the first 5–70% of the undisrupted makespan. The *dynamic obstacle density* ρ is the number of blockage events divided by the number of free cells. By default a generator filter keeps every instance solvable: blockages never disconnect the map or land on a goal, and emergency cells are reachable. Section 6 switches this filter off.

Communication. Robots within Manhattan distance $R = 6$ can exchange messages (ALERT, STATUS, REQUEST, PROPOSE, COMMIT, ACCEPT, REJECT, UPDATE). Messages are delivered within the time step, and every message is counted.

3 Method

3.1 Initial plan: prioritized Space-Time A*

Robots are planned one at a time, longest task chain first. Each path is written into a space-time reservation table (vertex and edge reservations, plus a parking reservation at the dock from arrival onwards). ST-A* searches states (x, y, g, t) , where g is the index of the next goal, so a robot’s whole pickup→delivery→...→dock chain is one search. The heuristic is the exact static distance through the remaining goals. If an attempt fails, the order is randomly restarted.

3.2 Local repair

When a disruption happens at time t , the robots whose reservations it invalidates are queued. Each queued robot a is repaired with the tiers below, cheapest in plans changed first. Let ℓ^* be a ’s robot-free (ideal) path, p_{solo} its best path with every other plan fixed, and $d(p) = |p| - |\ell^*|$ the delay. A coalition S of neighbours that replan jointly with a (with a planned first) is scored

$$\text{score}(S) = \underbrace{d(p_a) + \sum_{m \in S} \Delta_m}_{\text{extra steps}} + \lambda |S| + \text{DL}(p_a), \quad (1)$$

where Δ_m is member m 's added completion time, $\lambda = 3$ is the price of changing one more robot's plan, and DL adds a penalty of 10 if the plan makes an emergency delivery miss its deadline. A coalition is accepted only if its score is below that of going solo.

```

Repair(a, t):
   $\ell^* \leftarrow \text{ST-A*}(a, \text{ignoring other robots}); p_{\text{solo}} \leftarrow \text{ST-A*}(a, \text{others fixed})$ 
  if  $d(p_{\text{solo}}) \leq \delta$  and no deadline is lost then commit  $p_{\text{solo}}$ ; return // Tier 1
  for each radius  $r$  in  $(R, 2R [$ , wider ring if  $a$  has been holding]) do // Tiers 2-3
    ALERT robots within  $r$ ; candidates  $\leftarrow$  movable robots whose plans block  $\ell^*$ 
    for  $k = 1 \dots \text{max coalition size}$  do
      for each coalition  $S$  of  $k$  candidates do
        REQUEST/PROPOSE: replan  $a$  first, then each  $m \in S$ ;
          if  $m$  fails or  $\Delta_m > \delta$ :  $m$  recruits its own blockers (chain,  $\leq 2$  hops)
          if best score  $< \text{score}(\text{solo})$  then COMMIT best; REJECT the rest; return
  if  $a$  is urgent (emergency, or no solo path) then // Tier 3b
    group  $\leftarrow$  movable robots within  $2R$  crossing  $\ell^*$  ( $\leq 8$ )
    prioritized ST-A* for  $a + \text{group}$ ; accept if score  $< \text{score}(\text{solo})$ 
  if  $p_{\text{solo}}$  exists then commit  $p_{\text{solo}}$  // long detour
  else hold position, re-queue robots planned through  $a$ 's cell, retry next step // Tier 4

```

Design details.

- **Smallest coalition first.** Only robots that actually block a 's ideal path are candidates (at most 4). Sizes 1, 2, 3 are tried in order, and the search stops at the first size whose best coalition beats going solo. A size is skipped when $\lambda|S|$ alone already exceeds the solo delay.
- **Chains.** A member m that cannot dodge alone recruits up to 2 robots in its own range whose plans block its *relaxed* path (its best path if movable neighbours made way), at most 2 hops and 6 robots per tree. The whole tree is scored by Eq. (1), so a chain is chosen only if it beats both going solo and every flat coalition.
- **Tier 3b** is a bounded group replan around the stuck robot, not a global one. It is restricted to urgent robots because applying it to every stuck robot changed 0.6 more plans per disruption at 50 robots.
- **Emergencies.** At the event, the deadline is the robot-free delivery time plus 3 steps. Emergency robots negotiate for any delay ($\delta = 0$), may form coalitions of 4, and are never asked to move for others.
- **Permanent breakdown.** The broken robot's cell becomes an obstacle. Its unfinished tasks are auctioned to robots in range (lowest marginal cost wins), and a carried item is handed off at a reachable neighbouring cell.
- **Expanding ring.** A robot that has held for 10 steps doubles its search radius every 10 steps. This fixed a deadlock between two holding robots 16 cells apart (beyond $2R$), each needing the other's cell.

3.3 Strategies compared

- **local:** the method above (v2).
- **local-flat:** our first version (flat coalitions only, no Tier 3b, no deadline term).
- **local-nochain:** v2 without chains (robots sweep only).
- **solo:** Tier 1 only, an ablation with no negotiation.
- **full:** re-runs prioritized ST-A* for all live robots from the current state after every disruption. This is the reference that the assignment forbids for repair; it shows what staying local costs.

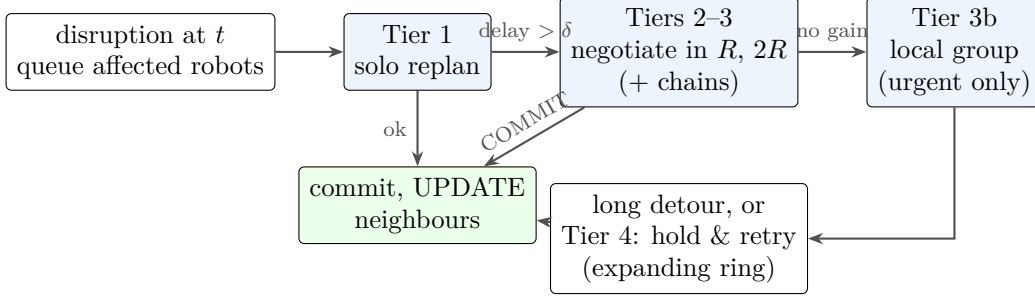


Figure 2: Repair protocol. Each tier is tried only if the previous one cannot do as well as going solo, so the robots touched grow only when needed.

4 Experimental setup

Every configuration was run for 10 seeds with every strategy on identical instances and event schedules. The defaults are 3 tasks per robot, 3% dynamic obstacle density (20% of blockages permanent), 2 breakdowns (30% permanent) and 2 emergencies per run.

- **Robots:** 5, 10, 20, 30, 40, 50.
- **Density:** $\rho = 0, 2, 5, 10, 15\%$ with 20 robots.
- **Single disruption:** exactly one disruption of each type (temporary or permanent blockage or breakdown, emergency) with 10–50 robots. This isolates the number of plans changed to handle one disruption.
- **Alter penalty:** $\lambda = 0, 1, 3, 6, 12$ at 20 and 40 robots.
- **Stress:** settings built to make robots fail (Section 6).

Altered counts the robots whose future path after the repair differs from the plan just before the disruption, including the disrupted robot and all cascades. **Direct** counts those the disruption itself invalidated. **Collateral** is altered minus direct. Every executed step is checked for vertex, edge and obstacle collisions, and the test suite (27 tests) re-validates all future plans after every disruption in a strict mode that forbids reservation overwrites.

5 Results

All 1,700 runs of the four main sweeps had zero collisions. Every task was completed in every run, except one **solo** run discussed in Section 6.

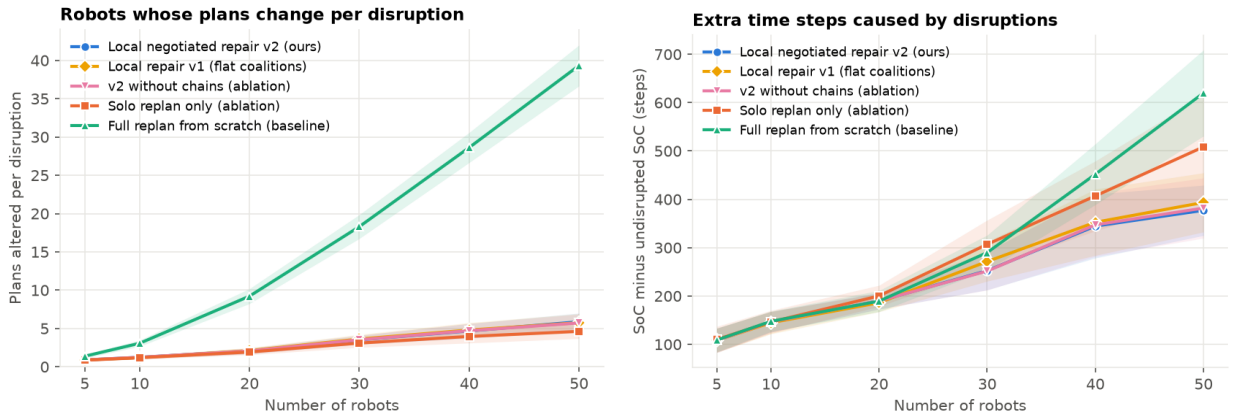


Figure 3: Robots sweep. Left: plans changed per disruption. Right: extra time steps caused by the disruptions. Bands are 95% confidence intervals.

Table 1: Robots sweep (means over 10 seeds; 12 disruptions per run).

robots	strategy	SoC	altered / dis.	collateral	emerg. on time	repair ms
20	local	2612	2.06	0.26	65%	126
	local-flat	2608	2.11	0.27	45%	81
	solo	2622	1.93	0.07	30%	12
	full	2611	9.22	7.28	100%	41
30	local	4093	3.52	0.77	25%	416
	local-flat	4112	3.62	0.71	20%	263
	solo	4148	3.09	0.25	20%	29
	full	4131	18.29	15.46	75%	124
50	local	6958	5.88	1.44	35%	2270
	local-flat	6974	5.75	1.42	15%	1102
	solo	7089	4.62	0.23	15%	67
	full	7201	39.33	35.09	100%	590

Plans changed. Local repair changes about one robot beyond those the disruption hits directly (collateral 0.3–1.4). Full replanning changes 9–39 robots per disruption, nearly the whole fleet, at no better total time: at 50 robots its SoC is *worse* (7201 vs 6958), because prioritized planning from scratch reshuffles priorities and delays robots that were not affected.

Total time. `local` has the lowest or near-lowest SoC at every size. Negotiation pays off as the fleet grows: `solo` needs 131 more steps than `local` at 50 robots, and its overhead rises fastest.

v2 vs v1. Compared with `local-flat`, v2 lowers SoC by 7–19 steps at 30–50 robots and raises the emergency on-time rate (65% vs 45% at 20 robots, 35% vs 15% at 50), changing about as many plans (+0.13 at 50 robots, fewer elsewhere). The cost is roughly twice the repair time. The `local-nochain` ablation is within noise of v2 at half the repair time: chains solve cases flat coalitions cannot (a covered test case), but such cases are rare in these sweeps.

Scaling. Repair time grows with fleet size, from milliseconds at 10 robots to about 2.3 s per disruption at 50 (Python, single core), mostly spent in ST-A* searches inside negotiation.

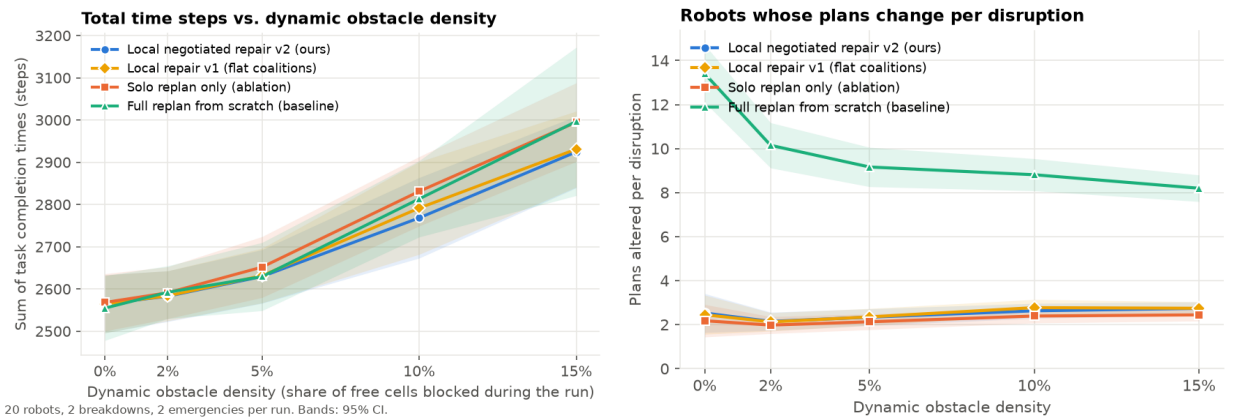


Figure 4: Density sweep (20 robots). SoC rises with ρ for all strategies. The plans changed per disruption stay flat for local repair and about 4 $\times$ higher for full replanning.

Obstacle density. From 0% to 15% density the number of disruptions per run grows from 4 to 45, and SoC overhead from 143 to 503 steps for `local`. Plans changed per disruption stay between 2.1 and 2.7. At 10% density v2 beats v1 on SoC (2768 vs 2792), plans changed (2.63 vs 2.77) and emergency on-time rate (40% vs 15%).

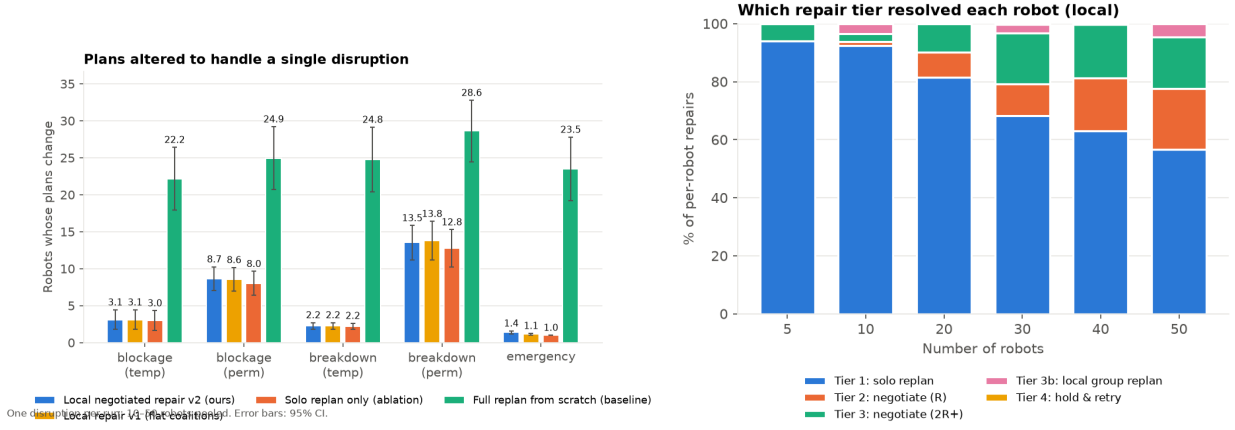


Figure 5: Left: plans changed to handle a single disruption, by type (10–50 robots pooled). Right: which tier resolved each robot’s repair.

Single disruption. A temporary breakdown changes 2.2 plans (exactly the robots hit directly) and a temporary blockage 3.1. Permanent events change more because their effects last: a permanent breakdown changes 13.5 plans, most of them direct (the robots that would pass the dead robot plus the winners of its tasks). Full replanning changes 22–29 robots for any single event. A solo replan (Tier 1) settles 94% of robot repairs at 5 robots but only 57% at 50: as the fleet grows, negotiation (Tiers 2–3) and Tier 3b take over a larger share. Holds stay below 0.3%.

Alter penalty. Figure 6 shows the trade-off set by λ : $\lambda = 0$ minimizes time but changes the most plans, and $\lambda = 12$ the opposite. At 40 robots the default $\lambda = 3$ for v2 is better than v1 at every λ on both axes.

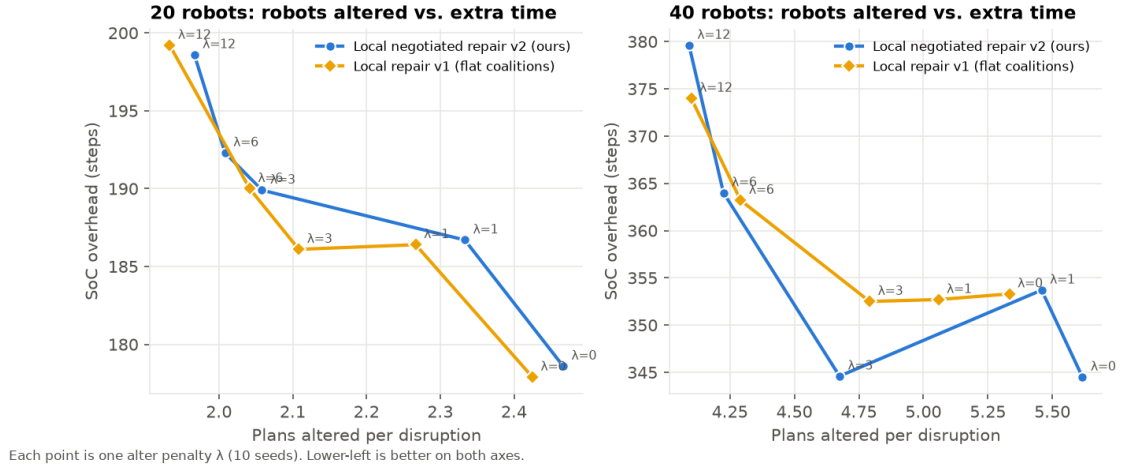


Figure 6: Plans changed vs. SoC overhead as the alter penalty λ varies.

6 Settings where agents fail

In the main sweeps robots almost never fail, but partly by construction: the scenario generator removes disruptions that would make a task impossible (Section 2). To find where the agents really fail, we ran a **stress study** (760 runs, 10 seeds, all strategies) in which this filter can be switched off and the environment pushed further.

- **unsafe**: filter off, half of the blockages and breakdowns permanent, $\rho = 0$ –20%, 20 robots;
- **unsafe-single**: one permanent blockage or breakdown with the filter off, 20 and 40 robots;

- **dense**: filter on, $\rho = 20\text{--}30\%$, 20 robots;
- **crowd**: a small 12×27 map (204 free cells, 46 docks) with 20–44 robots;
- **radius**: communication radius $R = 1, 2, 3, 6$ with 40 robots.

A run *fails* if some task is still undone when it ends. A run ends when every robot is done, or when no robot has moved and no goal has been reached for 150 steps. Each unfinished robot is classified as *goal cut off* (a remaining goal is unreachable past permanent obstacles and dead robots, so the instance has become unsolvable) or *deadlocked* (every goal is reachable, but the robot is stuck). No run had a collision.

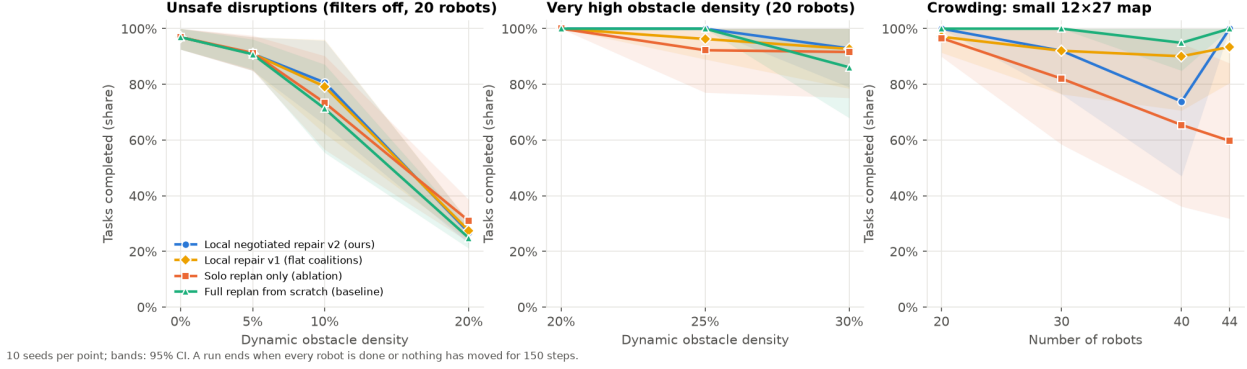


Figure 7: Share of tasks completed in the stress settings.

Table 2: Settings where agents fail. Runs failed out of 10 and the share of tasks completed; stuck robots per run for `local` (cut off / deadlocked).

setting	value	local (v2)		stuck cut / dead	local-flat completed	full completed
		failed	completed			
unsafe	$\rho = 0\%$	3/10	96.9%	0.5 / 0.2	96.9%	96.9%
	$\rho = 5\%$	7/10	91.0%	1.5 / 0.3	91.0%	90.8%
	$\rho = 10\%$	10/10	80.6%	2.3 / 3.1	79.2%	71.3%
	$\rho = 20\%$	10/10	26.8%	9.9 / 8.8	27.4%	24.8%
unsafe-single	1 perm. breakdown, 20 robots	2/10	97.5%	0.4 / 0.2	97.5%	97.5%
	1 perm. breakdown, 40 robots	4/10	96.7%	0.4 / 1.0	96.7%	96.7%
	1 perm. blockage, 20 or 40 robots	1/10	99.7%	0.1 / 0.0	99.7%	99.7%
dense	$\rho = 25\%$	0/10	100%	–	96.3%	100%
	$\rho = 30\%$	1/10	92.9%	0.0 / 2.0	92.7%	86.1%
crowd	30 robots	1/10	92.1%	0.0 / 2.9	92.1%	100%
	40 robots	3/10	73.9%	0.0 / 11.9	90.1%	94.9%
	44 robots	0/10	100%	–	93.4%	100%
radius	$R = 1, 2, 3, 6$	0/10	100%	–	100%	100%

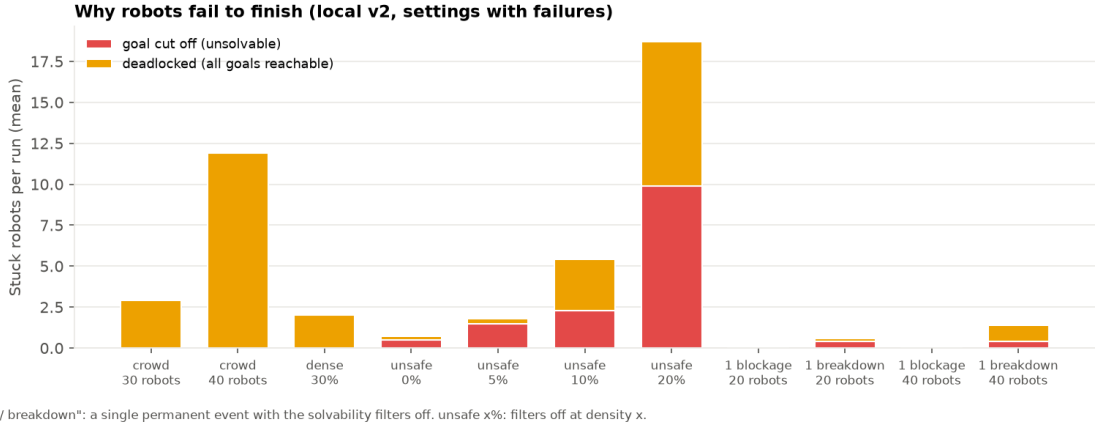


Figure 8: Why robots fail to finish under `local`: goals cut off (red) or deadlocked although every goal is reachable (orange).

F1. Disruptions that make tasks impossible

With the filter off, a permanent blockage or a robot that breaks down for good can cut off a pickup cell, a station or a whole aisle segment (Figure 9). No repair can complete those tasks, so all strategies fail alike. Completion falls from 96.9% at $\rho = 0\%$ to 26.8% at $\rho = 20\%$, where the map breaks into pieces and about 10 robots per run have a goal cut off. Even with no blockages at all ($\rho = 0\%$), 3 of 10 runs fail, because a permanently broken robot in a one-cell aisle is itself a wall. A *single* permanent breakdown is enough to make 2 of 10 runs fail at 20 robots and 4 of 10 at 40.

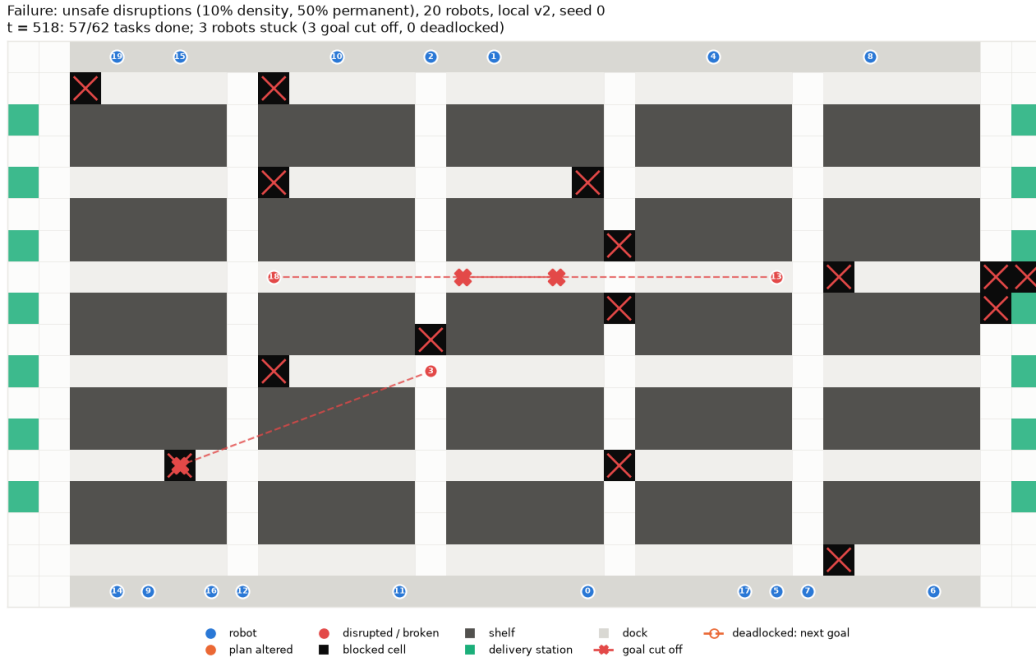


Figure 9: Failure F1/F2 (`results/demo_failure/unreachable.gif`). Permanent blockages (black) cut off the goals of robots 3, 13 and 18. They hold forever and 5 of 62 tasks are never done.

F2. Knock-on deadlock behind a stuck robot

A robot whose goal is cut off does not give up its task. It holds its cell indefinitely, and in one-cell aisles that cell can be the only way to another robot's goal. At $\rho = 10\%$, 2.3 robots per run have a goal cut off, but another 3.1 are deadlocked even though all of their goals are reachable. At 20% it is 9.9 and 8.8. More than half of the failures in these settings are therefore caused by the agents'

reaction, not by the instance. Local repair suffers less from this than full replanning (80.6% vs 71.3% completed at 10%, 3.1 vs 6.6 deadlocked robots), because a global replan rebuilds every plan around the stuck robots and can wall more robots in.

F3. Hold cascade in crowded maps (a weakness of our method)

On the small map with 40 robots, 3 of 10 `local` runs fail, and in each of them nearly the whole fleet stops. In seed 3, a robot breaks down *temporarily* in an aisle at $t = 14$ and directly blocks 6 robots. They cannot find a detour, so they hold (Tier 4). Every robot that planned through a held cell is then re-queued and also holds. Within one step all 40 robots hold, each seeing the others as permanent walls, and only 1 of 122 tasks is ever delivered. Negotiation cannot untangle this: coalitions have at most 3–4 members, and Tier 3b at most 8, while the knot involves 40. `full` replanning plans everyone together and solves the same instance (122/122). This is the setting in which the local method is clearly worse than the global baseline. `local-flat` failed in 1 of 10 runs here and `solo` in 4. The effect depends strongly on the instance: at 44 robots no `local` run failed, and the 95% confidence band in Figure 7 is wide.

F4. Very high obstacle density

With the filter on (every task stays solvable), failures start at about 25–30% density: 1 of 10 runs fails for each local strategy at 30% (2 deadlocked robots), and 2 of 10 for full replanning (86.1% completed). Up to 20% density no strategy failed.

F5. Gridlock without negotiation

Without negotiation (`solo`), robots gridlock much more often. On the main map with 50 robots, a single permanent breakdown at $t = 40$ leaves all 49 remaining robots holding from then on, and only 26 of 150 tasks are done (Figure 10). On the small map with 44 robots, 5 of 10 `solo` runs fail (59.7% completed). `local` solves both: 150/150 on the first instance and 10 of 10 runs on the second. Negotiation is what prevents these gridlocks.

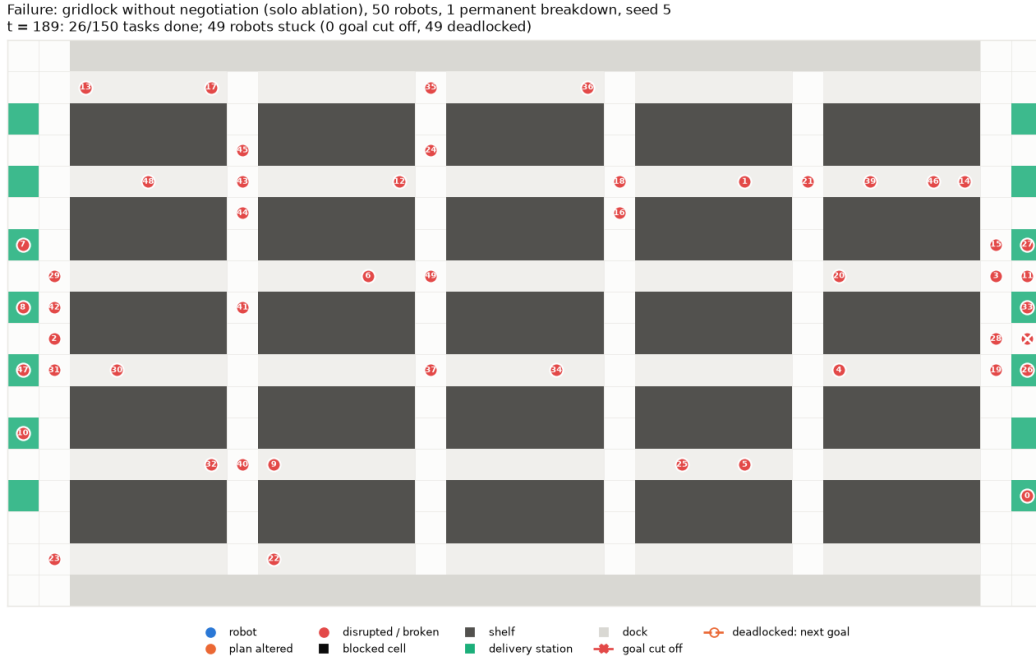


Figure 10: Failure F5 (`results/demo_failure/gridlock_solo.gif`): with `solo`, all 49 live robots (red) gridlock after one permanent breakdown. The same instance with `local` completes all 150 tasks.

F6. Emergency deadlines

Emergency tasks are always eventually delivered, but rarely by their deadline (ideal time + 3 steps) once the fleet is dense: 25–35% on time at 30–50 robots on the main map, and 5% on the crowded map with 40 robots and at 30% density. Full replanning reaches 70–100% because it can move every robot out of the emergency robot’s way.

Where the agents did not fail

- **Communication radius.** Shrinking R from 6 to 1 caused no failures at 40 robots and raised SoC by only 1% (5577 vs 5510). Short-range negotiation finds fewer partners, but robots that hold for 10 steps widen their search (expanding ring), which breaks the deadlocks a short radius would otherwise cause.
- **Solvable instances on the main map:** 5–50 robots and up to 20% obstacle density (1,700 runs of the main sweeps). No `local` run failed.

Summary

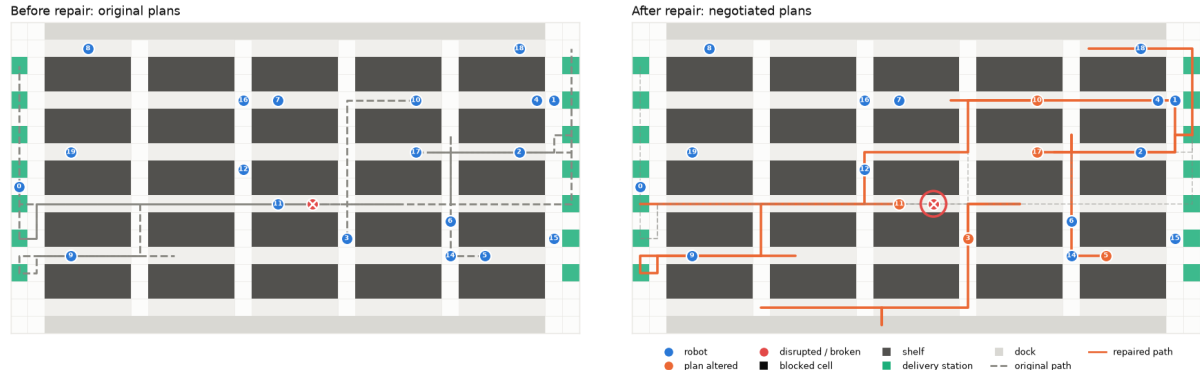
The agents fail to complete their tasks when (i) permanent disruptions cut goals off, even a single robot breaking down for good in a one-cell aisle; (ii) robots stuck in this way keep blocking others, which in these settings accounts for more than half of the stuck robots; (iii) a map is so crowded that one local hold cascades into a fleet-wide gridlock, the case where global replanning does better; (iv) obstacle density reaches about 30%; (v) robots cannot negotiate (`solo`). Possible remedies are to detect unreachable goals and release or re-auction the task while moving the robot aside, and to escalate to a larger regional replan once a hold cascade exceeds a few robots.

7 Graphical demonstration

All visuals are generated with matplotlib from the simulator’s run log by running `python experiments/make_demos.py`. They are:

- `results/demo/run.gif`: animation of a 20-robot run with 12 mixed disruptions. Altered robots and their repaired paths are highlighted for 8 steps after each event.
- `results/demo/replay.html`: interactive replay in the browser (play, pause, scrub).
- `results/demo_single/*.png`: before/after snapshots of each disruption type, with the old plan dashed and the repaired plan solid (Figure 11).
- `results/demo_failure/*`: GIFs and final-state snapshots of the failure cases in Section 6.
- `python -m warehouse_mapf.run -agents 20 -live`: replays any run in an interactive window.

Disruption #0 at t=55: robot 13 breaks down permanently
agents directly affected: 5 plans altered: 6 additional (negotiated) changes: 1 tiers used: [1, 3]



(a) permanent breakdown

Disruption #0 at t=92: robot 5 gets emergency task (pickup (13, 8))
agents directly affected: 1 plans altered: 2 additional (negotiated) changes: 1 tiers used: [2]



(b) emergency task

Figure 11: Disruption snapshots from the demonstration: the dashed line is the plan before the repair, the solid line the repaired plan, and orange robots are those whose plans changed.

8 Limitations and future work

- **Prioritized planning is incomplete.** Neither the initial planner nor the joint replans inside negotiation guarantee a solution when one exists. Holds and the expanding ring recover in practice, but not provably.
- **Robots never give up a task.** A robot whose goal is permanently cut off holds forever. It should detect this, report it, and release or re-auction the task.
- **Emergency deadlines are soft.** The deadline term changes which repair is preferred, but it cannot preempt robots already in the way. In dense fleets most deadlines (ideal + 3 steps) are missed.
- **Repair time** grows to seconds at 50 robots in Python. Caching searches, bounding chains (they add about $2\times$ for little gain) or a compiled ST-A* would help.
- **Idealized communication:** instant, lossless messages within range. Delays and dropped messages are not modelled.

Reproducing

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q tests # 27 tests
.venv/bin/python experiments/run_experiments.py # main sweeps (~20 min, 10 cores)
.venv/bin/python experiments/run_experiments.py --sweep stress
.venv/bin/python -m warehouse_mapf.viz.plots # figures + results/summary.md
.venv/bin/python experiments/make_demos.py # GIFs, snapshots, replay
```

Code: <https://github.com/SreenityaThatikunta/warehouse-plan-repair>